Génie électrique et informatique industrielle

université de BORDEAUX

Compte-rendu : Automatisation d'un banc de mesures sous Python

PREVOST-BONNEFILLE Louison
KouthieyeThiaw

Groupe : B1

I-Introduction..... 3
II-Caractérisation des Appareils..... 3
III-Programmation des différentes appareils..... 4
1. Identification des ressources via Python.....5
IV- Schéma de câblage et Montage..... 6
V- Automatisation des mesures..... 6
1. Mesure ponctuelle..... 6
2. Balayage automatique (Sweep).....9
VI- Résultats et Analyse..... 11
Conclusion.....11

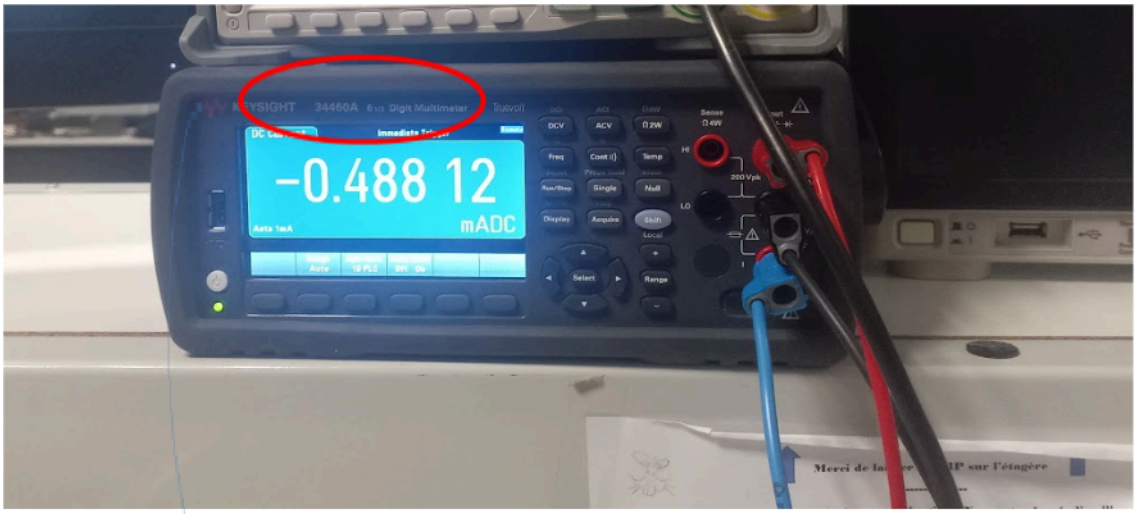
I-Introduction

Dans le cadre de notre BUT GEII , on a eu à concevoir un programme python qui va nous servir a travers un oscilloscope à envoyer des scripts python afin de de pouvoir relever des valeurs de courant et de tension .Pour ce faire on a dû pendant 3 séances appliquer une méthodologie rigoureuse passant par lecture des datasheets de nos appareils de mesures afin de comprendre leur fonctionnement et les différentes commandes.

II-Caractérisation des Appareils

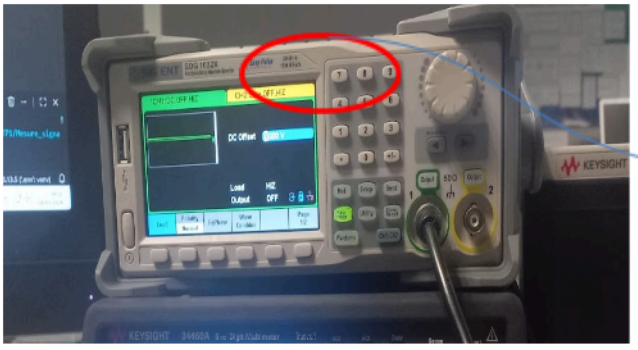
1-Multimetre

Le premier travail que nous avons fait est tout d'abord d'identifier nos appareils de mesures (GBF et multimètre) pour connaître leurs caractéristiques.
Ainsi dans notre banc de TP, nous avions les appareils ci-dessous :



Grace a cette information, on a pu déduire le mode de notre multimètre qui est comme le montre l'image le modèle KEYSIGHT 34460A.

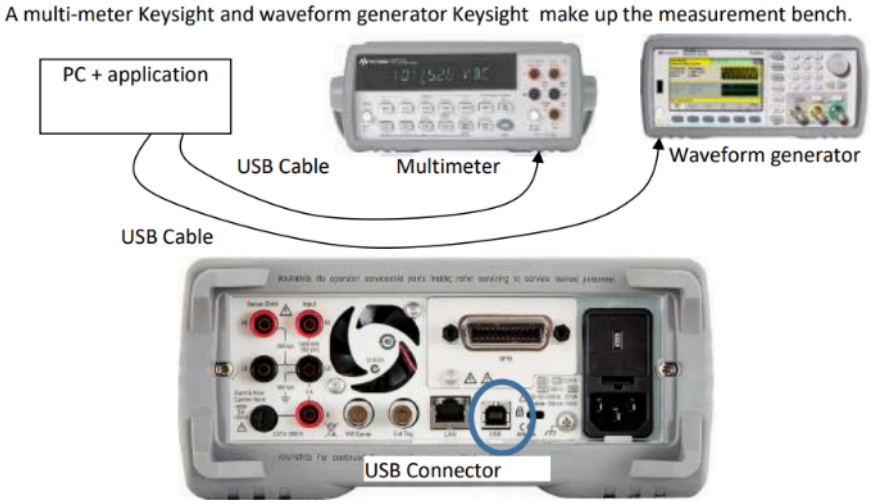
Après avoir identifié notre modèle de multimètre , une simple recherche sur internet nous permis de retrouver les information clefs qui nous serviront plutard a la programmation de notre appareil via l'IDE Vscode en python afin de relever les différentes mesures.
2- GBF

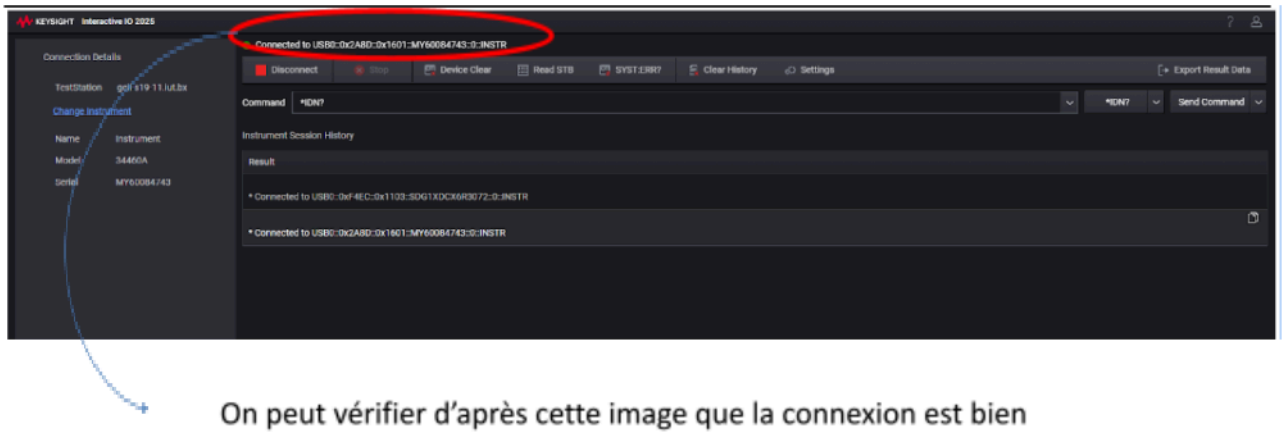


De manière analogue au multimètre 334460A , on a effectué le même travail pour notre générateur SIG ENT SDG1032X

III-Programmation des différentes appareils

Après avoir pris connaissance de nos différentes appareils utilisés, on va passer a la partie programmation via l'IDE Vscode. Pour cela nous allons suivre comme indiqué dans le TP les différentes étapes.





On peut vérifier d'après cette image que la connexion est bien établie. On peut aussi voir l'IDN de nos appareils ainsi que des paramètres clefs qui nous serviront dans le code.

Après avoir connecté nos appareils via USB, nous avons utilisé le logiciel **Keysight Interactive IO 2025** pour vérifier que la connexion était bien établie et opérationnelle. Cette interface nous a permis de visualiser l'**IDN** de chaque instrument ainsi que les paramètres de configuration essentiels pour l'élaboration de nos scripts Python.

1. Identification des ressources via Python

Pour débuter la programmation, nous utilisons la bibliothèque pyvisa. Le "Resource Manager" permet de lister tous les instruments connectés au bus USB.

Code identification des appareils :

```
import pyvisa
rm = pyvisa.ResourceManager('@py')
print(rm)


rm = pyvisa.ResourceManager()
print(rm)


liste_appareils = rm.list_resources()


print(f'Appareils trouvés : {liste_appareils}')
```

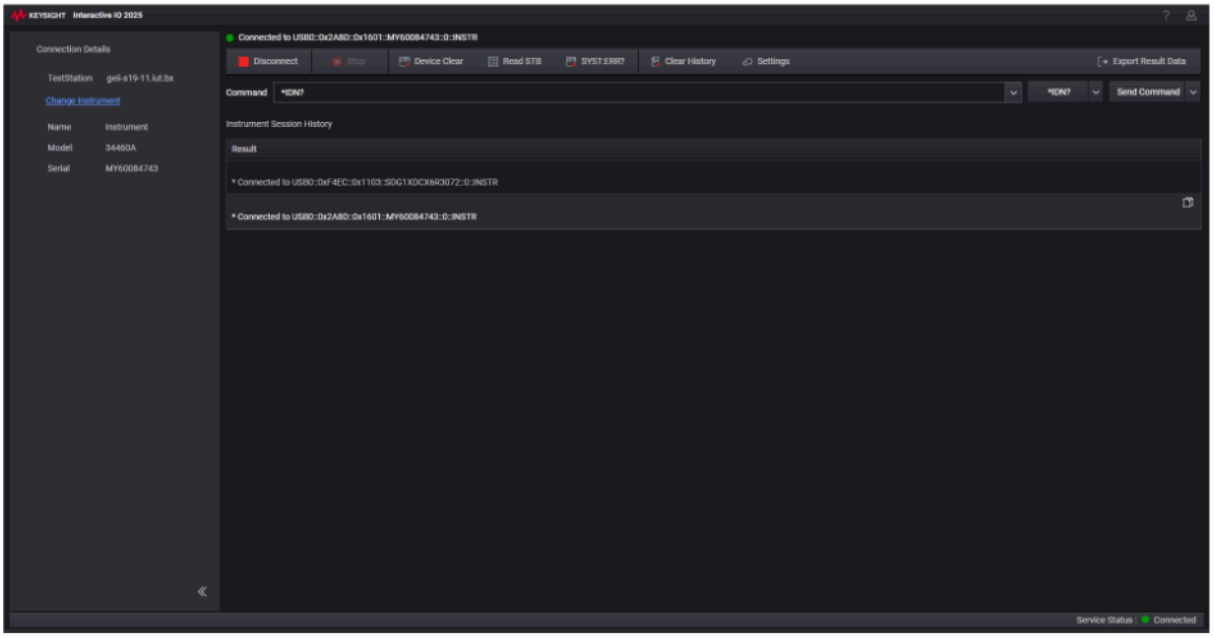


figure 1 : liste des appareils disponibles sur keysight

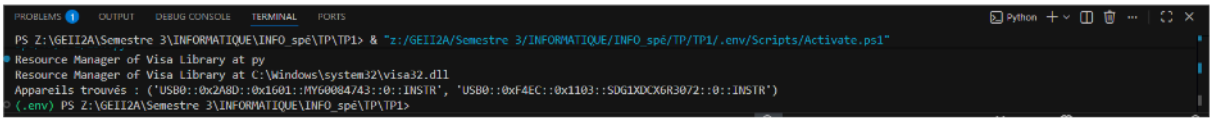


figure 2 : Liste correctement les appareils disponibles sur le terminal

GBF : USB0::0xF4EC::0x1103::SDG1XDCX6R3072::0::INSTR
Multimètre : USB0::0x2A8D::0x1601::MY60084743::0::INSTR

IV- Schéma de câblage et Montage

Pour obtenir la caractéristique Id(V) de la diode, nous avons réalisé un montage incluant une résistance de protection de **470 Ω**. Ce montage permet de mesurer précisément le courant traversant le composant et la tension à ses bornes.

- **Mesure de l'intensité (Id)** : Le multimètre est configuré en mode ampèremètre (bornes 10A ou 3A selon l'appareil) et placé en série avec la diode.
- **Mesure de la tension (Vd)** : Le multimètre est configuré en mode voltmètre DC et branché en parallèle aux bornes de la diode.
- **Générateur (GBF)** : Il est utilisé en mode "DC Offset" pour appliquer une tension continue variable.

V- Automatisation des mesures

1. Mesure ponctuelle

Nous avons d'abord validé la communication en effectuant une lecture unique de la tension et du courant pour vérifier la cohérence des branchements.

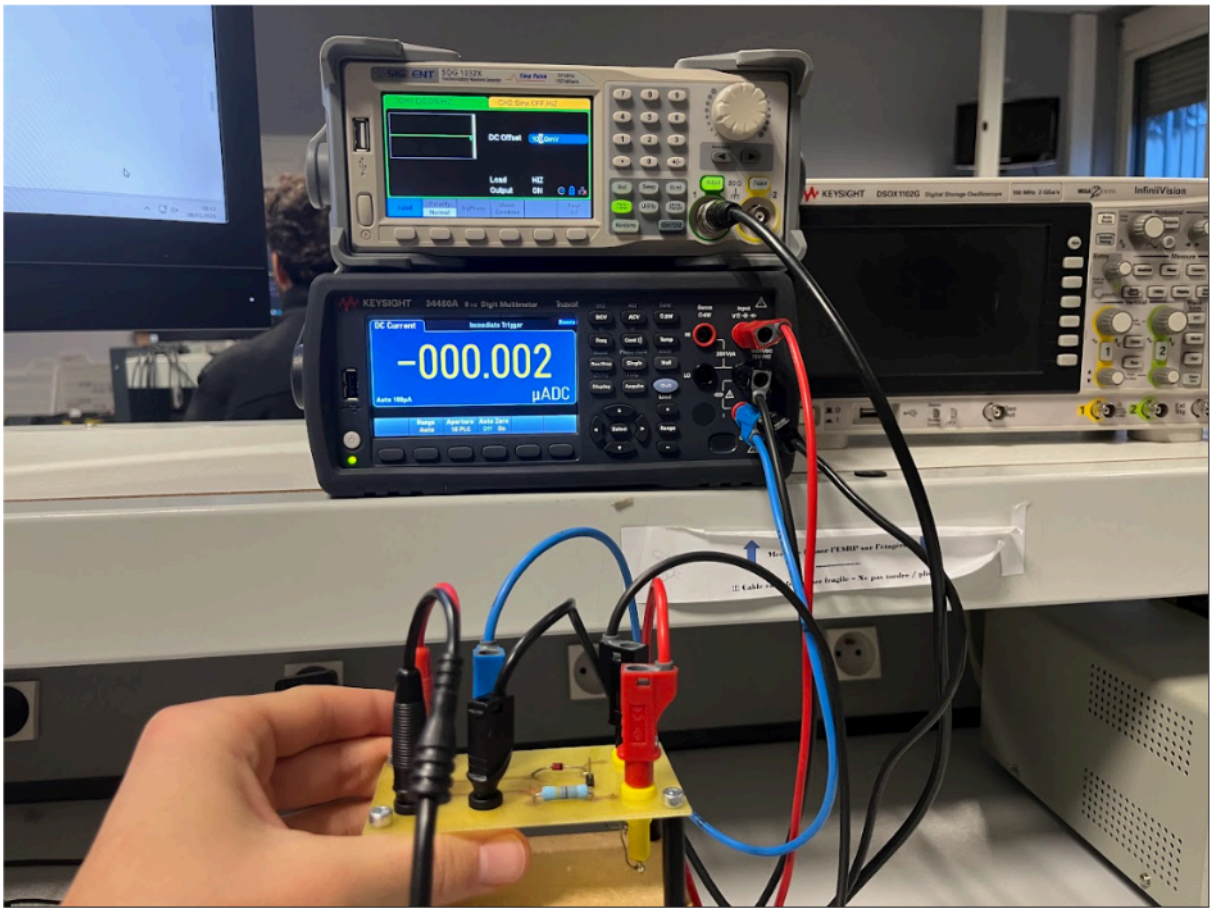


figure 3 : Montage mesure tension/courant LED

Mesure simple :

```
import pyvisa

import time




rm = pyvisa.ResourceManager()


#PARTIE GENERATION SIGNAL GBF
```

```
adresse_gbf = 'USB0::0xF4EC::0x1103::SDG1XDCX6R3062::0::INSTR'



gbf = rm.open_resource(adresse_gbf)



commande_signal = 'C1:BSWV WVTP,DC,OFST,1'

gbf.write(commande_signal)

print("Signal configuré : 100mV, Continu")



gbf.write('C1:OUTP ON,LOAD,HZ')

print("Sortie Canal 1 : ACTIVÉE (High-Z)")



#PARTIE MULTIMETRE

adresse_multi = 'USB0::0x2A8D::0x1601::MY60084743::0::INSTR'



multi = rm.open_resource(adresse_multi)

mesure_u = multi.query('MEAS:VOLT:DC?')

mesure_c = multi.query('MEAS:CURR:DC?')

print(f"Résultat tension : {mesure_u}")

print(f"Résultat courant : {mesure_c}")
```

2. Balayage automatique (Sweep)

Pour tracer la courbe complète, nous avons automatisé la variation de tension du générateur. Nous avons utilisé un balayage de 0 à 1V avec un pas de 0.05V pour obtenir une précision suffisante autour du seuil de la diode.

Balayage et tracé :

```
import pyvisa
import time
import numpy as np
import matplotlib.pyplot as plt

rm = pyvisa.ResourceManager()

adresse_gbf = 'USB0::0xF4EC::0x1103::SDG1XDCX6R3072::0::INSTR'
gbf = rm.open_resource(adresse_gbf)

adresse_multi = 'USB0::0x2A8D::0x1601::MY60084743::0::INSTR'
multi = rm.open_resource(adresse_multi)

tensions_envoyees = np.linspace(0, 1, 20)
u_mesures = []
i_mesures = []

gbf.write('C1:OUTP ON,LOAD,HZ')

for v in tensions_envoyees:
    gbf.write(f'C1:BSWV WVTP,DC,OFST,{v}')
    time.sleep(2)

    val_u = float(multi.query('MEAS:VOLT:DC?'))
    val_i = float(multi.query('MEAS:CURR:DC?'))

    u_mesures.append(val_u)
    i_mesures.append(-1*1000*val_i)

gbf.write('C1:OUTP OFF')
gbf.close()
multi.close()
```

```
plt.figure()
plt.plot(u_mesures, i_mesures, '-o')
plt.title('Caractéristique I = f(U)')
plt.xlabel('Tension mesurée (V)')
plt.ylabel('Courant mesuré (A)')
plt.grid(True)
plt.show()
```

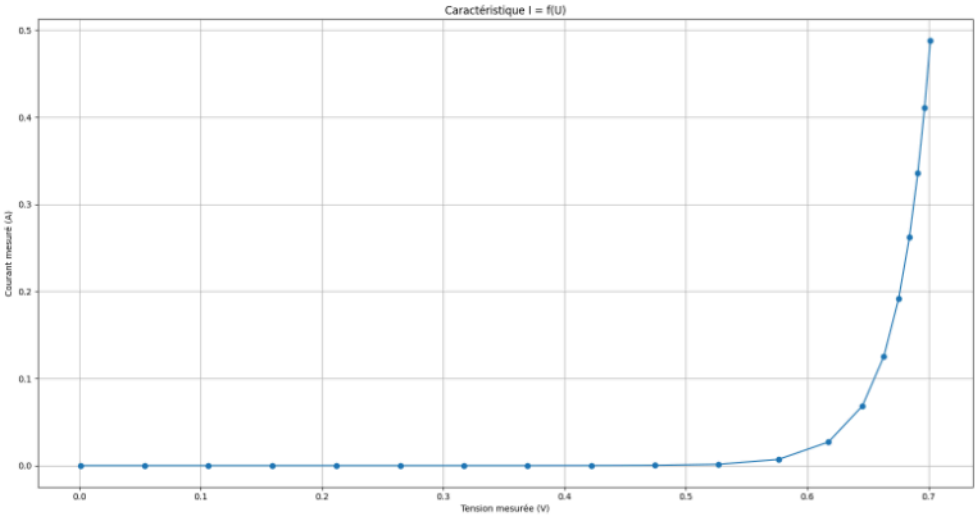


figure 4 : Tracé du courant I en fonction de la tension V

Vidéo fonctionnement balayage de tension :

IMG_3058.MOV

VI- Résultats et Analyse

Le programme génère un graphique représentant le courant en fonction de la tension.

Analyse de la courbe obtenue :

- **Tension de seuil** : Le courant reste quasi nul jusqu'à environ **0,6 V**, ce qui correspond à la tension de seuil typique d'une diode au silicium.
- **Zone passante** : Au-delà du seuil, le courant augmente de façon exponentielle avec la tension.
- **Correction de polarité** : Nous avons multiplié les valeurs de courant par **-1** dans le code car l'ampèremètre était branché de façon à donner des valeurs négatives en convention récepteur.

Conclusion

Ce TP nous a permis de maîtriser l'automatisation d'un banc de mesure complexe. L'utilisation de scripts Python facilite grandement la récolte de données massives et permet de caractériser des composants électroniques avec une précision et une répétabilité impossibles à atteindre manuellement. Nous avons pu piloter avec succès des appareils de marques différentes (Keysight et Siglent) au sein d'un même programme.